

	SR UNIVERSITY	
	Campus Warangal	
	Program: II - B. Tech (CS& AI)	Batch: 66
	Name: Shaishab Saha	Reg. No: 2503A51688
	Department: Computer Science and AI	Semester: I
	AI Assisted Coding-Assignment	Date: 22-09-2026

1. Question

What would happen if the hierarchy contained 10,000 departments? Ask AI to explain the efficiency of searching in different tree structures.

2. Prompt

Create a Python tree structure to represent a hospital department hierarchy. Implement adding child nodes, displaying the hierarchy, searching for a department, and counting nodes. Explain how a tree is different from a list and provide test cases.

3. Code

```

4. """Hospital department hierarchy represented with a tree.
5.
6. A tree stores data in parent-child levels, so one department can contain
7. subdepartments. A list stores items in a single linear sequence and does not
8. represent these relationships directly.
9. """
10.
11. class Department:
12.     """A node in the hospital department tree."""
13.
14.     def __init__(self, name):
15.         self.name = name
16.         self.children = []
17.
18.         def add_child(self, department):
19.             """Add a Department node below this department."""
20.             if not isinstance(department, Department):
21.                 raise TypeError("department must be a Department")
22.             self.children.append(department)

```

```

23.
24.     def display(self, level=0):
25.         """Print this department and all descendants with indentation.""" 26.         print(" " * level +
self.name)
27.         for child in self.children:
28.             child.display(level + 1)
29.
30.     def search(self, name):
31.         """Return the first department with the given name, or None."""
32.         if self.name.lower() == name.lower():
33.             return self
34.         for child in self.children:
35.             result = child.search(name)
36.             if result is not None:
37.                 return result 38.         return None
39.
40.     def count_nodes(self):
41.         """Return the number of nodes in this subtree, including this node."""
42.         return 1 + sum(child.count_nodes() for child in self.children)
43.
44.     def build_hospital_tree():
45.         """Build a sample hospital hierarchy for demonstration and testing."""
46.         hospital = Department("Hospital")
47.         clinical = Department("Clinical Services")
48.         administration = Department("Administration")
49.         hospital.add_child(clinical) 50.         hospital.add_child(administration)
51.
52.         emergency = Department("Emergency")
53.         surgery = Department("Surgery")
54.         clinical.add_child(emergency)
55.         clinical.add_child(surgery)
56.         surgery.add_child(Department("Operating Rooms"))
57.         administration.add_child(Department("Human Resources")) 58.         return hospital
59.
60.     if __name__ == "__main__":
61.         # Test case 1: displaying the hierarchy.
62.         hospital = build_hospital_tree()
63.         print("Hospital hierarchy:")
64.         hospital.display()
65.
66.         # Test case 2: searching (search is case-insensitive).

```

```

67.     assert hospital.search("emergency").name == "Emergency"
68.     assert hospital.search("Radiology") is None
69.     print("Search tests passed.")
70.
71.     # Test case 3: counting all nodes.
72.     assert hospital.count_nodes() == 7
73.     assert hospital.search("Surgery").count_nodes() == 2 74.     print("Node-count tests passed.")
75.
76.     # Test case 4: adding a new child changes the total count.
77.     hospital.search("Clinical
    Services").add_child(Department("Radiology"))
78.     assert hospital.count_nodes() == 8
79.     print("Add-child test passed.")

```

4. Output

Hospital hierarchy:

Hospital

 Clinical Services

 Emergency

 Surgery

 Operating Rooms

 Administration

 Human Resources

Search tests passed.

Node-count tests passed.

Add-child test passed.

5. Observation

The task demonstrates how a tree data structure can represent a hospital's hierarchical department structure. It supports adding child departments, displaying the hierarchy, searching for departments, and counting total departments.

TASK-2

1. QUESTION:

Why is a graph more suitable than a simple list for representing connections between hospital locations?

2.PROMPT:

Create a Python graph structure to represent hospital locations and their connections. Add nodes and edges, display the graph using an adjacency list, check whether two locations are connected, and find routes using BFS or DFS. Test the program with at least three source-destination pairs and explain why a graph is suitable for representing hospital connections.

3.CODE:

```
from collections import deque
```

```
class HospitalGraph:
```

```
    def __init__(self):
```

```
        self.graph = {}
```

```
        def add_location(self, location):
```

```
            self.graph.setdefault(location, [])
```

```
            def add_connection(self, first, second):
```

```
                self.add_location(first)
```

```
                self.add_location(second)        if second
```

```
                not in self.graph[first]:
```

```
                    self.graph[first].append(second)        if
```

```
                    first not in self.graph[second]:
```

```
                        self.graph[second].append(first)
```

```
            def display(self):
```

```
                for location, connections in self.graph.items():
```

```
                    print(f'{location}: {', '.join(connections) or 'None'}')
```

```

def find_route(self, source, destination, method="BFS"):
    if source not in self.graph or destination not in self.graph:
        return None
    pending = deque([source])
    if method == "BFS":
        visited = {source}
        previous = {source: None}
        while pending:
            current = pending.popleft() if method == "BFS" else pending.pop()
            if current == destination:
                route = []
                while current is not None:
                    route.append(current)
                    current = previous[current]
                return route[::-1]
            for neighbor in self.graph[current]:
                if neighbor not in visited:
                    visited.add(neighbor)
                    previous[neighbor] = current
                    pending.append(neighbor)
        return None

```

```

def connected(self, source, destination):
    return self.find_route(source, destination) is not None

```

```

hospital = HospitalGraph()
locations = ("Emergency", "Radiology", "Pharmacy",
            "Laboratory", "ICU", "Clinic")
for location in locations:
    hospital.add_location(location)

```

```

for first, second in (
    ("Emergency", "Radiology"),
    ("Emergency", "Pharmacy"),

```

```

    ("Radiology", "Laboratory"),
    ("Laboratory", "ICU"),
    ("Pharmacy", "Clinic"),
):
    hospital.add_connection(first, second)

print("Hospital adjacency list:") hospital.display()

print("\nConnectivity and routes:") tests =
(
    ("Emergency", "ICU", "BFS"),
    ("Clinic", "Laboratory", "DFS"),
    ("Emergency", "Clinic", "BFS"),
)
for source, destination, method in tests:
    route = hospital.find_route(source, destination, method)
    print(f'{source} -> {destination} ({method}): connected={route is not None},
route={route}')

```

Graphs are suitable because hospital locations are nodes and corridors or
roads are edges. They represent many-to-many connections and support #
connectivity checks and route finding through BFS or DFS.

4.OUTPUT:

Hospital adjacency list:

Emergency: Radiology, Pharmacy

Radiology: Emergency, Laboratory

Pharmacy: Emergency, Clinic

Laboratory: Radiology, ICU

ICU: Laboratory

Clinic: Pharmacy

Connectivity and routes:

Emergency -> ICU (BFS): connected=True, route=['Emergency', 'Radiology', 'Laboratory', 'ICU']

Clinic -> Laboratory (DFS): connected=True, route=['Clinic', 'Pharmacy', 'Emergency', 'Radiology', 'Laboratory']

Emergency -> Clinic (BFS): connected=True, route=['Emergency', 'Pharmacy', 'Clinic']

5.OBSERVATION:

The graph successfully represents hospital locations as nodes and their connections as edges. BFS/DFS can be used to check connectivity and find routes between different hospital locations.

TASK-3:

1.QUESTION:

Implement a Priority Queue using a Heap to manage emergency patients according to their severity. Add patients with priority values, view and remove the highest-priority patient, display waiting patients, handle priority ties, and test insertion and removal operations.

2.PROMPT:

Create a Python program using a Heap-based Priority Queue for an emergency department. Assign priorities 1 to 4, where 1 is Critical, 2 is Serious, 3 is Moderate, and 4 is Normal. Implement operations to add patients, view the highest-priority patient, remove the highest-priority patient, display waiting patients, handle patients with equal priority appropriately, and test insertion and removal.

3.CODE:

```
"""Emergency department priority queue implemented with a min-heap."""
```

```
import heapq from dataclasses
```

```
import dataclass
```

```
PRIORITIES = {1: "Critical", 2: "Serious", 3: "Moderate", 4: "Normal"}
```

```
@dataclass(frozen=True)
```

```
class Patient:    name: str
```

```
priority: int
```

```
def __str__(self) -> str:
```

```
    return f'{self.name} - Priority {self.priority} ({PRIORITIES[self.priority]}')
```

```
class EmergencyDepartment:    def
```

```
    __init__(self):
```

```
        self._heap = []
```

```
self._arrival_order = 0
```

```
    def add_patient(self, name: str, priority: int) -> None:
```

```
    if priority not in PRIORITIES:        raise
```

```
ValueError("Priority must be 1, 2, 3, or 4.")    patient =
```

```
Patient(name, priority)
```

```
    # Arrival order makes equal-priority patients FIFO.
```

```
    heapq.heappush(self._heap, (priority, self._arrival_order, patient))
```

```
self._arrival_order += 1
```

```
    def view_highest_priority(self):
```

```
    return self._heap[0][2] if self._heap else None
```

```
def remove_highest_priority(self):
```

```
    return heapq.heappop(self._heap)[2] if self._heap else None
```

```
def display_waiting_patients(self):
```

```
    return [entry[2] for entry in sorted(self._heap)]
```

```
def test_priority_queue() -> None:
    department = EmergencyDepartment()
    department.add_patient("Alice", 3)
    department.add_patient("Bob", 1)
    department.add_patient("Cara", 2)
    department.add_patient("Dan", 1) # Bob is treated before Dan.
    print("Waiting patients:")
    for patient in department.display_waiting_patients():
        print(patient)
```

```
    print("\nHighest-priority patient:")
    print(department.view_highest_priority())
```

```
    print("\nRemoval order:")
    while department.view_highest_priority() is not None:
        print(department.remove_highest_priority())
```

```
if __name__ == "__main__":
```

```
    test_priority_queue()
```

4.OUTPUT: AI

ASSISTED LAB/T3.py"

Waiting patients:

Bob - Priority 1 (Critical)

Dan - Priority 1 (Critical)

Cara - Priority 2 (Serious)

Alice - Priority 3 (Moderate)

Highest-priority patient: Bob

- Priority 1 (Critical)

Removal order:

Bob - Priority 1 (Critical)

Dan - Priority 1 (Critical)

Cara - Priority 2 (Serious) Alice

- Priority 3 (Moderate)

5.OBSERVATION:

The Heap-based Priority Queue successfully manages emergency patients according to their priority levels. Higher-priority patients are served first, while patients with the same priority are handled appropriately.

TASK-4:

1.QUESTION:

Compare searching medical words using a list with searching using a Trie. Explain the advantages and disadvantages.

2.PROMPT:

Create a Python Trie data structure for storing medical words and drug names. Insert medical terms into the Trie, search for an exact word, check whether a prefix exists, and

display/autocomplete all words beginning with a given prefix. Test the program using at least three different prefixes.

3.CODE:

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_word = False

class MedicalTrie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for character in word.lower().strip():
            node = node.children.setdefault(character, TrieNode())
        node.is_word = True

    def _find_node(self, text):
        node = self.root
        for character in text.lower().strip():
            if character not in node.children:
                return None
            node = node.children[character]
        return node
```

```

def search(self, word):
    node = self._find_node(word)
    if node is not None and node.is_word:
        return [word]

def starts_with(self, prefix):
    return self._find_node(prefix) is not None

def autocomplete(self, prefix):
    prefix = prefix.lower().strip()
    node = self._find_node(prefix)
    if node is None:
        return []

    words = []

    def collect(current, suffix):
        if current.is_word:
            words.append(prefix + suffix)
        for character in sorted(current.children):
            collect(current.children[character], suffix + character)

    collect(node, "")

    return words

if __name__ == "__main__":
    medical_words = [

```

```
"aspirin", "amoxicillin", "amlodipine", "antibiotic",  
"acetaminophen", "bronchitis", "bradycardia", "diabetes",  
"diagnosis", "doxycycline", "paracetamol", "penicillin",  
]
```

```
trie = MedicalTrie() for  
word in medical_words:  
    trie.insert(word)  
print("Exact search:")  
print("aspirin:",  
      trie.search("aspirin"))  
print("aspirins:",  
      trie.search("aspirins"))
```

```
print("\nPrefix tests:") for  
prefix in ("am", "br", "do"):  
    print(f'{prefix!r} exists: {trie.starts_with(prefix)}')  
print("Autocomplete:", trie.autocomplete(prefix))
```

4.OUTPUT:

AI ASSISTED

LAB/T4.py" Exact

search: aspirin: True

aspirins: False

Prefix tests:

'am' exists: True

Autocomplete: ['amlodipine', 'amoxicillin']

'br' exists: True

Autocomplete: ['bradycardia', 'bronchitis']

'do' exists: True

Autocomplete: ['doxycycline']

5.OBSERVATION:

The Trie successfully stores and searches medical words efficiently. It supports exact word searching, prefix checking, and autocomplete, making it useful for medical applications that require quick term suggestions.

TASK-5:

1.QUESTION:

For each hospital-software scenario, choose the most appropriate non-linear data structure — Tree, Graph, Heap, or Trie — and explain why it is suitable. Compare your choices with an AI recommendation.

2.PROMPT: For each given hospital-software scenario, select the most suitable data structure from Tree, Graph, Heap, or Trie. Explain the reason for each choice, identify the strengths and weaknesses, and compare the choices with an AI recommendation.

3.CODE:

""Hospital data-structure recommendations and an AI comparison.""

SCENARIOS = [

{

"scenario": "Patient triage",

"choice": "Heap",

"why": "Keeps the most urgent patient available without sorting the whole queue.",

"operation": "peek/extract highest priority",

```

    "complexity": "peek O(1); insert/extract O(log n)",
  },
  {
    "scenario": "Hospital departments and referral routes",
    "choice": "Graph",
    "why": "Models departments as vertices and referrals, travel, or transfers as edges.",
    "operation": "BFS/DFS traversal",
    "complexity": "O(V + E)",
  },
  {
    "scenario": "Organization of wards and subwards",
    "choice": "Tree",
    "why": "Represents the natural parent-child hierarchy and supports subtree queries.",
    "operation": "search/insert in a balanced tree",
    "complexity": "O(log n) average/guaranteed when balanced",
  },
  {
    "scenario": "Fast lookup of medical codes or drug prefixes",
    "choice": "Trie",
    "why": "Shares common prefixes and supports autocomplete efficiently.",
    "operation": "prefix lookup",
    "complexity": "O(L), where L is the key length",
  },
]

```

```
def print_recommendations():
```

```

for item in SCENARIOS:

    print(f'{item['scenario']}: {item['choice']}')

print(f' Why: {item['why']}')    print(f' Main
operation: {item['operation']}')    print(f'
Typical Big-O: {item['complexity']}')


    print("\nAI recommendation comparison:")
print(

    "The AI recommendation matches these choices: Heap for urgency, Graph for "
    "relationships, Tree for hierarchy, and Trie for prefix-based lookup. "
    "The best choice depends on the dominant operation; for example, a general "
    "queue could be simpler than a Heap if priorities never change."
)

if __name__ == "__main__":
    print_recommendations()

```

4.OUTPUT:

AI ASSISTED LAB/T5.py"

Patient triage: Heap

Why: Keeps the most urgent patient available without sorting the whole queue.

Main operation: peek/extract highest priority

Typical Big-O: peek $O(1)$; insert/extract $O(\log n)$

Hospital departments and referral routes: Graph

Why: Models departments as vertices and referrals, travel, or transfers as edges.

Main operation: BFS/DFS traversal

Typical Big-O: $O(V + E)$

Organization of wards and subwards: Tree

Why: Represents the natural parent-child hierarchy and supports subtree queries.

Main operation: search/insert in a balanced tree

Typical Big-O: $O(\log n)$ average/guaranteed when balanced

Fast lookup of medical codes or drug prefixes: Trie

Why: Shares common prefixes and supports autocomplete efficiently.

Main operation: prefix lookup

Typical Big-O: $O(L)$, where L is the key length

AI recommendation comparison:

The AI recommendation matches these choices: Heap for urgency, Graph for relationships, Tree for hierarchy, and Trie for prefix-based lookup. The best choice depends on the dominant operation; for example, a general queue could be simpler than a Heap if priorities never change.

5.OBSERVATION:

The activity shows that different hospital applications require different data structures based on their relationships and operations. Tree, Graph, Heap, and Trie are useful for hierarchical data, connected locations, priority-based processing, and prefix-based searching respectively.

Scenario	Your Choice + Reason
Hospital department hierarchy	Tree+A Tree represents hierarchical relationships between departments and sub-departments.
Finding a route between hospital rooms	Graph+A Graph represents locations as nodes and connections/routes as edges. Emergency patient priority based on severity
Emergency patients based on severity	Heap+A Heap efficiently manages patients according to priority, serving the highest-priority patient first.

Autocomplete medical terms	Trie+A Trie is designed for fast prefix searching and autocomplete of medical words.
Frequently finding the highest-priority task	Graph+A Graph can represent multiple connections and relationships between different hospital units.

Student Activity:

//Choose the data structure: Hospital department hierarchy-Tree

//Explain why it is appropriate: Tree: Represents hierarchical parent-child relationships.

//Identify the main operation: Tree → Traversal / Search

//State the expected/typical Big-O complexity where appropriate: Tree → Search: $O(\log n)$ for a balanced search tree

//Ask AI for a recommendation : “For these hospital applications—department hierarchy, room route finding, emergency patient priority, medical-word autocomplete, and highest-priority task retrieval—recommend the most suitable data structure from Tree, Graph, Heap, and Trie. Explain the reason and typical Big-O complexity for each.”

//Compare your choice with AI's recommendation: The AI recommendations are consistent with my choices. Tree is suitable for hierarchy, Graph for routes and connections, Heap for priority-based tasks, and Trie for medical-word autocomplete.

//If you disagree with AI, explain why:

I do not disagree with the AI recommendation because the selected data structures match the main operations required by each hospital application.